Automated LLM/Agent-Guided Compiler Code Optimization

Two-page technical explanation for project demonstration and viva

What the project does

This project is a compiler-design system that accepts a C program, converts it into intermediate code, finds safe optimization opportunities, verifies every proposed change, and accepts a change only when the program still behaves identically and its measured weighted cost becomes lower. It combines classical compiler optimization with an optional local language-model assistant. The language model can suggest a safe pass order, but it never has authority to commit code changes.

End-to-end pipeline

```
C source -> lexical analysis -> parsing and AST -> symbol table and semantic analysis ->
intermediate code -> analysis -> optimization -> verification -> cost evaluation -> final output.
```

The project already contains a lexer, recursive-descent parser, AST nodes, symbol-table support, and semantic checks. The executable intermediate representation is LLVM IR. A separate TAC-style display is generated for academic clarity: TAC presents operations in simple forms such as *t1 = a + b*, while LLVM IR additionally models types, basic blocks, memory operations, branches, loops, and calls.

Core agent components

Component	Responsibility
Orchestrator	Runs analyze -> propose -> verify -> evaluate -> accept/reject until no beneficial opportunity remains.
Code Analysis Specialist	Finds constant pairs, algebraic patterns, dead results, repeated subexpressions, arithmetic operations, basic blocks, and loop information.
Optimization Specialist	Uses compiler passes in rule mode, or uses optional local LLM guidance to prioritize available safe passes.
Verification Module	Compiles baseline and candidate programs and performs differential testing on 13 inputs.
Cost Evaluator	Compares a weighted score based on instructions, expensive arithmetic, temporary values, and measured runtime.

How a candidate is accepted

For each optimization pass, the orchestrator creates a copy of the current LLVM IR. It never modifies the accepted version directly. The candidate copy is compiled. If compilation fails, it is rejected. If it compiles, the baseline and candidate executables are run using one default case, edge cases, and reproducible random inputs. The verifier compares standard output, exit codes, and timeouts. Any mismatch causes rejection.

```
Weighted cost = instruction count + 3 x expensive arithmetic operations + temporary register
count + 10 x mean execution time in milliseconds.
```

Therefore, the acceptance rule is: **accept only if output verification passes AND candidate cost is lower.** This fail-closed design ensures that an incorrect or unhelpful transformation is not committed.

Optimization Methods and Syllabus Connection

The project demonstrates standard code-optimization concepts from compiler design. Constant propagation replaces known variables with their values. Constant folding evaluates expressions such as *10 + 20* during compilation. Strength reduction replaces costly arithmetic, for example *x * 8* with a left shift. Common-subexpression elimination avoids repeated calculations. Dead-code elimination removes values that do not affect output. Loop-invariant code motion can move calculations outside loops when their values do not change per iteration.

For code-generation topics, the final LLVM IR can be lowered into a short target-assembly excerpt. For intermediate-code topics, the program can show TAC-style views before and after optimization. This connects the project to lexical analysis, parsing, symbol tables, semantic analysis, intermediate-code generation, basic blocks, control flow, data-flow analysis, loop optimization, machine-independent optimization, and target-code generation.

Example

A program may contain: *int a = 10 + 20;*, *int unused = 5 * 4;*, and repeated calculations such as *4 * 8*. The compiler can fold constants to 30 and 32, remove the unused assignment, and simplify intermediate operations. The original and optimized programs must still print the same value, for example *Result: 94*.

Interactive Demonstration

When the main program runs, the user can choose to paste C code, run a small dataset demo, or run the full dataset. For pasted code, the program optionally asks whether to use local LLM guidance and whether to display educational TAC and assembly views. The concise terminal result shows accepted or rejected transformations, original output, optimized output, number of matching tests, final PASS or FAIL status, cost reduction, and an optimized IR excerpt.

Dataset, Metrics, and Reports

The project automatically generates 500 benchmark programs: arithmetic-only, nested expressions, repeated expressions, dead code, loops, conditionals, and mixed programs. For every program it records the decision history, final cost, final IR, output-match verdict, and convergence steps. Aggregate reports include verification pass rate, acceptance rate, average cost reduction, convergence distribution, LLM validity rate when enabled, and false-positive rate. Generated charts include a decision-flow Sankey diagram, per-optimization bar chart, iteration-cost line chart, optimization pie chart, steps-versus-improvement scatter plot, and category heatmap.

Key Viva Statement

This is a compiler-design project enhanced with an optional intelligent assistant. The core logic remains compiler based: intermediate-code analysis, data-flow and loop analysis, classical optimization passes, differential testing, cost measurement, and safe code generation. The LLM is only an advisor; correctness and cost evaluation decide every final optimization.